CSE110: Object Oriented Programming
Section: 10
Spring 24 Semester

Project Report
Football Management System

Course Code	: CSE110
Course Title	: Object Oriented Programming
Section	:10
Group No	:1
Group Name	:Abstract Thinkers

Submitted by:

Student ID	Student Name	Contribution Percentage
2023-3-60-057	Nusrat Rahman Aurna	25%
2023-3-60-070	Sadia hasan Munmun	23%
2023-3-60-076	Zeba Tahira	22%
2023-3-60-028	Abdullah Al Usaeed Chowdhury	30%

Submitted To:
Ahmed Abdal Shafi Rasel,
Lecturer, Department of CSE
East West University
Submission Date: 18. 01. 2025

Table of Contents

1 Introduction **1**

2 Objective **2**

3 Methodology **3**

3.1 Problem Design (UML Class Diagram)

4 Implementation **4**

5 Reflection on Individual and Team work **6**

6 Limitations and Future Work **7**

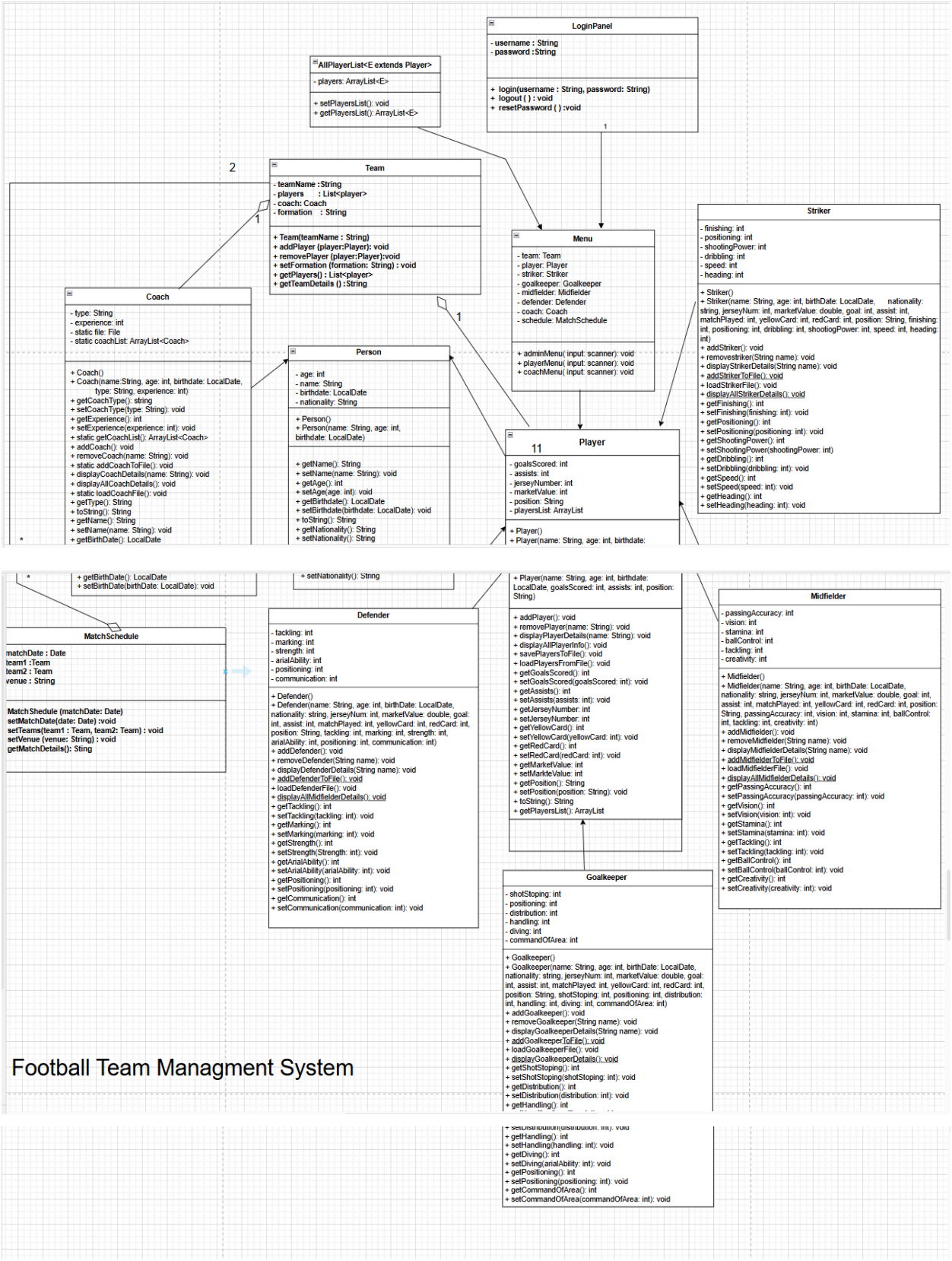
7 References **8**

Introduction

A Football Management System is a Java-based application for managing football teams, leagues, and player statistics. It includes functionalities like team roster management, match scheduling, player performance tracking, and league standings. Designed for coaches and managers, it leverages Java's object-oriented features (classes, inheritance, polymorphism) and integrates databases for data storage and retrieval.

Objective

The Football Management System streamlines team and league management by organizing player details, tracking performance, generating reports, and monitoring league standings. It provides an efficient, user-friendly platform to enhance team management and administrative tasks.


LoginPanel
- username : String
- password :String
+ login(username : String, password: String)
+ logout ( ) : void
+ resetPassword ( ) :void
AllPlayerList<E extends Player>
- players: ArrayList<E>
+ setPlayersList(): void
+ getPlayersList(): ArrayList<E>
Team
- teamName :String
- players : List<player>
- coach: Coach
- formation : String
+ Team(teamName : String)
+ addPlayer (player:Player): void
+ removePlayer (player:Player):void
+ setFormation (formation: String) : void
+ getPlayers() : List<player>
+ getTeamDetails () :String
Menu
- team: Team
- player: Player
- striker: Striker
- goalkeeper: Goalkeeper
- midfielder: Midfielder
- defender: Defender
- coach: Coach
- schedule: MatchSchedule
+ adminMenu(input: scanner): void
+ playerMenu(input: scanner): void
+ coachMenu(input: scanner): void
Striker
- finishing: int
- positioning: int
- shootingPower: int
- dribbling: int
- speed: int
- heading: int
+ Striker()
+ Striker(name: String, age: int, birthDate: LocalDate, nationality: string, jerseyNum: int, marketValue: double, goal: int, assist: int, matchPlayed: int, yellowCard: int, redCard: int, position: String, finishing: int, positioning: int, dribbling: int, shootiogPower: int, speed: int, heading: int)
+ addStriker(): void
+ removestriker(String name): void
+ displayStrikerDetails(String name): void
+ addStrikerToFile(): void
+ loadStrikerFile(): void
+ displayAllStrikerDetails(): void
+ getFinishing(): int
+ setFinishing(finishing: int): void
+ getPositioning(): int
+ setPositioning(positioning: int): void
+ getShootingPower(): int
+ setShootingPower(shootingPower: int)
+ getDribbling(): int
+ setDribbling(dribbling: int): void
+ getSpeed(): int
+ setSpeed(speed: int): void
+ getHeading(): int
+ setHeading(heading: int): void
Coach
- type: String
- experience: int
- static file: File
- static coachList: ArrayList<Coach>
+ Coach()
+ Coach(name:String, age: int, birthdate: LocalDate, type: String, experience: int)
+ getCoachType(): string
+ setCoachType(type: String): void
+ getExperience(): int
+ setExperience(experience: int): void
+ static getCoachList(): ArrayList<Coach>
+ addCoach(): void
+ removeCoach(name: String): void
+ static addCoachToFile(): void
+ displayCoachDetails(name: String): void
+ displayAllCoachDetails(): void
+ static loadCoachFile(): void
+ getType(): String
+ toString(): String
+ getName(): String
+ setName(name: String): void
+ getBirthDate(): LocalDate
+ setBirthDate(birthDate: LocalDate): void
Person
- age: int
- name: String
- birthdate: LocalDate
- nationality: String
+ Person()
+ Person(name: String, age: int, birthdate: LocalDate)
+ getName(): String
+ setName(name: String): void
+ getAge(): int
+ setAge(age: int): void
+ getBirthdate(): LocalDate
+ setBirthdate(birthdate: LocalDate): void
+ toString(): String
+ getNationality(): String
+ setNationality(): String
Player
- goalsScored: int
- assists: int
- jerseyNumber: int
- marketValue: int
- position: String
- playersList: ArrayList
+ Player()
+ Player(name: String, age: int, birthdate: LocalDate, goalsScored: int, assists: int, position: String)
+ addPlayer(): void
+ removePlayer(name: String): void
+ displayPlayerDetails(name: String): void
+ displayAllPlayerInfo(): void
+ savePlayersToFile(): void
+ loadPlayersFromFile(): void
+ getGoalsScored(): int
+ setGoalsScored(goalsScored: int): void
+ getAssists(): int
+ setAssists(assists: int): void
+ getJerseyNumber: int
+ setJerseyNumber: int
+ getYellowCard(): int
+ setYellowCard(yellowCard: int): void
+ getRedCard(): int
+ setRedCard(redCard: int): void
+ getMarketValue: int
+ setMarkteValue: int
+ getPosition(): String
+ setPosition(position: String): void
+ toString(): String
+ getPlayersList(): ArrayList
MatchSchedule
matchDate : Date
team1 :Team
team2 : Team
venue : String
MatchShedule (matchDate: Date)
setMatchDate(date: Date) :void
setTeams(team1 : Team, team2: Team) : void
setVenue (venue: String) : void
getMatchDetails(): Sting
Defender
- tackling: int
- marking: int
- strength: int
- arialAbility: int
- positioning: int
- communication: int
+ Defender()
+ Defender(name: String, age: int, birthDate: LocalDate, nationality: string, jerseyNum: int, marketValue: double, goal: int, assist: int, matchPlayed: int, yellowCard: int, redCard: int, position: String, tackling: int, marking: int, strength: int, arialAbility: int, positioning: int, communication: int)
+ addDefender(): void
+ removeDefender(String name): void
+ displayDefenderDetails(String name): void
+ addDefenderToFile(): void
+ loadDefenderFile(): void
+ displayAllMidfielderDetails(): void
+ getTackling(): int
+ setTackling(tackling: int): void
+ getMarking(): int
+ setMarking(marking: int): void
+ getStrength(): int
+ setStrength(Strength: int): void
+ getArialAbility(): int
+ setArialAbility(arialAbility: int): void
+ getPositioning(): int
+ setPositioning(positioning: int): void
+ getCommunication(): int
+ setCommunication(communication: int): void
Midfielder
- passingAccuracy: int
- vision: int
- stamina: int
- ballControl: int
- tackling: int
- creativity: int
+ Midfielder()
+ Midfielder(name: String, age: int, birthDate: LocalDate, nationality: string, jerseyNum: int, marketValue: double, goal: int, assist: int, matchPlayed: int, yellowCard: int, redCard: int, position: String, passingAccuracy: int, vision: int, stamina: int, ballControl: int, tackling: int, creativity: int)
+ addMidfielder(): void
+ removeMidfielder(String name): void
+ displayMidfielderDetails(String name): void
+ addMidfielderToFile(): void
+ loadMidfielderFile(): void
+ displayAllMidfielderDetails(): void
+ getPassingAccuracy(): int
+ setPassingAccuracy(passingAccuracy: int): void
+ getVision(): int
+ setVision(vision: int): void
+ getStamina(): int
+ setStamina(stamina: int): void
+ getTackling(): int
+ setTackling(tackling: int): void
+ getBallControl(): int
+ setBallControl(ballControl: int): void
+ getCreativity(): int
+ setCreativity(creativity: int): void
Goalkeeper
- shotStoping: int
- positioning: int
- distribution: int
- handling: int
- diving: int
- commandOfArea: int
+ Goalkeeper()
+ Goalkeeper(name: String, age: int, birthDate: LocalDate, nationality: string, jerseyNum: int, marketValue: double, goal: int, assist: int, matchPlayed: int, yellowCard: int, redCard: int, position: String, shotStoping: int, positioning: int, distribution: int, handling: int, diving: int, commandOfArea: int)
+ addGoalkeeper(): void
+ removeGoalkeeper(String name): void
+ displayGoalkeeperDetails(String name): void
+ addGoalkeeperToFile(): void
+ loadGoalkeeperFile(): void
+ displayGoalkeeperDetails(): void
+ getShotStoping(): int
+ setShotStoping(shotStoping: int): void
+ getDistribution(): int
+ setDistribution(distribution: int): void
+ getHandling(): int
+ setHandling(handling: int): void
+ getDiving(): int
+ setDiving(arialAbility: int): void
+ getPositioning(): int
+ setPositioning(positioning: int): void
+ getCommandOfArea(): int
+ setCommandOfArea(commandOfArea: int): void
Football Team Managment System

UML Diagram

Implementation:

1. Login Panel

Role-based access login method.

```
***************************************************
*                                                 *
*   WELCOME TO FOOTBALL TEAM MANAGEMENT SYSTEM    *
*                                                 *
***************************************************


 ______________
|--------------|
|  1. Player   |
|--------------|
|  2. Coach    |
|--------------|
|  3. Admin    |
|--------------|
Enter 0 to Exit System.
Enter Your Role: 1
```

2. Menu

Role-based menu after login.

```
Welcome to Player Menu

Main Menu:
1. Display Player Details.
2. Display All Player Details.
3. Display Striker Details.
4. Display All Striker Details.
5. Display Midfielder Details.
6. Display All Midfielder Details.
7. Display Defender Details.
8. Display All Defender Details.
9. Display Goalkeeper Details.
10. Display All Goalkeeper Details.
11. Display Match Schedule.
12. Exit.
Enter choice: 12

        Exiting Player Menu...
```

```
Welcome to Coach Menu.

Main Menu:
1. Add Player.
2. Remove Player.
3. Display Coach Details.
4. Assign Coach to Team.
5. Remove Coach from Team.
6. Display Full Team.
7. Add Player to Team.
8. Remove Player from Team.
9. Exit.
Enter choice: 9

        Exiting Coach Menu...
```

```
Welcome to Admin Menu

Main Menu:
1. Add Player.
2. Remove Player.
3. Add Coach.
4. Remove Coach.
5. Display Full Team.
6. Add New Match Schedule.
7. Display Match Schedule.
8. Exit.
Enter choice: 8

        Exiting Admin Menu...
```

3. Player Menu

A player is only allowed to view information of players birthdate, age, nationality, jersey number, goal scored, assists, match played, red and yellow card, position etc.
1.Striker(finishing, positioning,shooting power, dribbling, speed, heading)
2. Midfielder(passing accuracy,vision,stamina,ball control,tackling,creativity)
3.Defender(tackling,marking,strength,aerial ability,communication)
4.Goalkeeper(shot stopping,distribution,handling,diving,command of area)

```
Enter choice: 2
Striker's name: Lionel Messi
Age: 36
Birth Date: 1987-06-24
nationality: Argentina
jersey Number: 10
Goal Scored: 800
Assist: 300
Match Played: 1000
Yellow Card: 5
Red Card: 0
Position: RW
Finishing: 95
Positioning: 92
Shooting Power: 88
Dribbling: 94
Speed: 89
Heading: 85
```

4. Coach Menu

A coach can add different types of players (e.g., Striker, Midfielder) with details like name, age, nationality, and statistics such as goals, assists, and matches played.

They can also input performance attributes (e.g., finishing, speed) and track disciplinary records (yellow/red cards).

```
Enter choice: 1
Which type of player you want to add?
1.Striker
2.Midfielder
3.Defender
4.Goalkeeper
Enter choice: 1
Adding a Striker...
Enter Striker name: nusrat
Enter Striker age: 23
Enter the Striker's Birth Date(yy-MM-dd): 02-12-01

Enter the Striker Nationality: Bangladeshi
Enter the Striker Jersey Number: 23
Enter the Striker goal scored: 12
Enter the Striker Assist: 5
Number of match the striker played: 13
Yellow card number: 4
Red Card number: 1
Enter the position: 9
Finishing: 8
Positioning: 5
Shooting Power: 9
Dribbling: 3
Speed: 90
Heading: 6
Player Added successfully.
```

5. Admin Menu

An admin can add, edit, or delete players and coaches, ensuring all details are updated correctly. They can also manage team composition, assign coaches, and oversee player and coach information for smooth team operations.

```
Enter choice: 3
Enter Coach name: Jürgen Klopp
Enter coach age:
57
Enter the Coach's Birth Date(yy-MM-dd):
67-06-16

Enter the Coach's Nationality:
Germany
Enter the coach type: Motivator
Enter the coach experience(in years): 18
coach added Successfully
```

Reflection on Individual and Team work:

Nusrat Rahman Aurna:

- Worked on the UML diagram.
- Developed the JAVA class.
- Assisted with debugging.
- Contributed to writing the report.
- Error finding.

Abdullah Al Usaeed Chowdhury:

- Work on the UML diagram
- Developed the JAVA class
- Marge All JAVA class then Run
- Error and Exception finding.

Sadia Hasan Munmun:

- Designed the basic structure.
- Worked on the UML diagram.
- Contributed to writing the report.
- Created the JAVA class.
- Helped debug the code.

Zeba Tahira:

- Worked on the UML diagram.
- Created the JAVA class.
- Contributed to writing the report.
- Helped debug the code.

Team Work:

- Exchanged ideas, discussed agreements and disagreements.
- Voted to make final decisions.
- Divided tasks based on strengths for efficient report writing.
- Ensured open communication and collaboration.
- Reviewed progress regularly to stay aligned with project goals.

Limitations and Future Work:

1. Advanced data analysis.
2. Enhanced user interface.
3. Simultaneous multi-user interactions.
4. Management of multiple teams.

References:

1. Java File Input/Output (Youtube) : https://bit.ly/3VgD7eo

2. SampleProject: https://github.com/Kazi-Ferdous-Hasan/College-Management-System